



Towards enterprise-specific question-answering for IT operations and maintenance based on retrieval-augmented generation mechanism

Zhuoxuan Jiang^{a,*}, Tianyang Zhang^b, Shengguang Bai^b, Lin Lin^c, Haotian Zhang^b, Yinong Xun^b, Jiawei Ren^b, Wen Si^a, Shaohua Zhang^a

^a Shanghai Business School, Shanghai, China

^b Learnable.ai, Shanghai, China

^c UCloud Technology Co., Ltd., Shanghai, China

ARTICLE INFO

Keywords:

IT operations and maintenance
Question-answering system
Large language model
Retrieval augmented generation

ABSTRACT

In the field of IT operations and maintenance, Question-Answering (QA) systems are very important for fault diagnosis, employee training, knowledge management and other aspects of providing high-quality services. While Large Language Models (LLMs) have significantly enhanced the performance of open-domain QA, the efficient handling of enterprise-exclusive corpora and the development of domain-specific high intelligent QA systems (e.g., QA for troubleshooting) remain underexplored in application settings. This paper proposes a comprehensive methodology based on Retrieval Augmented Generation (RAG) to facilitate the entire process of establishing such QA systems for IT operations and maintenance. Our methodology consists of two main components: (1) Offline Models Fine-tuning & Data Vectorization and (2) Online QA System Process. In the first component, we design a general data preprocessing pipeline and employ a contrastive learning method with two negative sampling strategies, i.e., Homogeneous In-Batch Sampling (HIS) and Auxiliary Hard Negative Sampling (AHNS), to fine-tune the embedding model. We also design instruction templates to fine-tune the LLM using Continue Pre-Training (CPT) and Retrieval Augmented Fine-Tuning (RAFT). In the second component, we develop an efficient RAG-based QA system process for online deployment. We gather enterprise-specific data from the domain of cloud computing, and our extensive experiments on the data demonstrate that the proposed methods outperforms counterparts on various QA tasks which have different difficulty levels. The experiments also illustrate a practical application of the proposed methodology in real-world enterprise-level scenarios.

1. Introduction

In recent years, IT operations and maintenance have become increasingly important due to the rapid growth of massive data and complex IT systems, such as those in cloud computing and telecommunications (Liu et al., 2024). Efficient IT operations and maintenance are essential for ensuring high-quality performance, reliability, and security for customers in the business sector (Du et al., 2017; Guo et al., 2024).

Traditionally, operating and maintaining these systems has relied heavily on IT operators' personal experience, often leading to challenges in incident management, problem resolution, and maintaining service quality (Jäntti & Cater-Steel, 2017). With advancements in Question-Answering (QA) techniques, some QA systems have been developed, enabling IT operators to retrieve useful information or efficiently create troubleshooting plans through natural-language human-machine inter-

action (Galup et al., 2009; Huang et al., 2023b; Jäntti & Cater-Steel, 2017). As shown in Fig. 1, two typical and important QA tasks are *Knowledge Acquisition* and *Troubleshooting* (Rijal et al., 2022). The former typically supports junior IT operators in building experience, while the latter provides senior operators with guidance for resolving complex software and hardware issues in their daily work. Consequently, QA systems have become increasingly important in modern IT operations and maintenance.

To build QA systems for IT operations and maintenance, we examined numerous examples, such as those shown in Fig. 1. The main characteristics and challenges are summarized as follows: First, QA utterances contain extensive technical terminology (e.g., status codes, service names, and underlined words ([*]) as shown in Fig. 1), with meanings specific to a particular domain or even just within an enterprise. Thus, enterprise-specific semantics must be carefully modeled. Second, a vast

* Corresponding author.

E-mail addresses: jzx@sbs.edu.cn (Z. Jiang), tianyang.zhang@learnable.ai (T. Zhang), shengguang.bai@learnable.ai (S. Bai), lesley.lin@ucloud.cn (L. Lin), haotian.zhang@learnable.ai (H. Zhang), yinong.xun@learnable.ai (Y. Xun), jiawei.ren@learnable.ai (J. Ren), siw@sbs.edu.cn (W. Si), zhangsh@sbs.edu.cn (S. Zhang).

<https://doi.org/10.1016/j.eswa.2025.130961>

Received 18 November 2024; Received in revised form 12 December 2025; Accepted 23 December 2025

Available online 29 December 2025

0957-4174/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

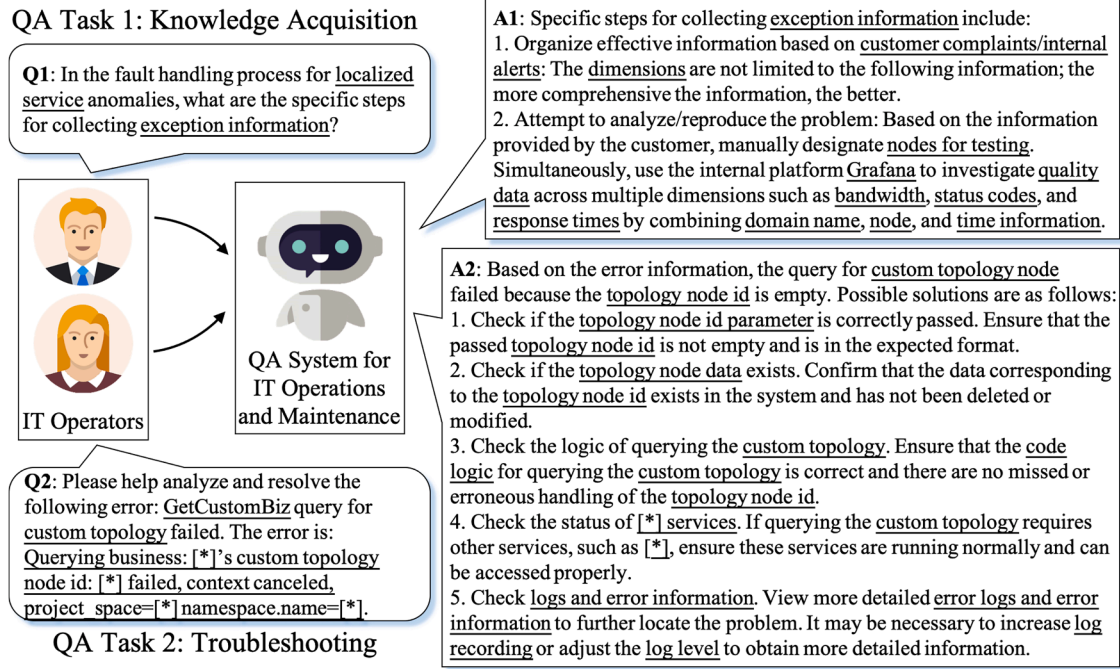


Fig. 1. Two examples of typical and important QA scenarios for the domain of IT operations and maintenance. The words with underlines are domain-specific terminologies, and the [*] represents enterprise-exclusive terms, e.g. status codes and service names.

array of enterprise-specific documents, guides, and manuals must be processed and converted into a standardized format to support ongoing QA system updates. Third, different QA tasks vary in complexity. For example, knowledge acquisition requires the system to provide straightforward answers, while troubleshooting requires the system to generate longer, summarized solutions based on multiple sources. Addressing these challenges leads to a complex question: how to build enterprise-specific QA systems using a generalized and efficient approach that handles exclusive data and varied QA tasks effectively.

Intuitively, open-domain QA systems trained on large public corpora can be further fine-tuned on domain-specific corpora and tasks, especially with recent advancements in open-source LLMs (Achiam et al., 2023; Brown et al., 2020; Chowdhery et al., 2023), including LLaMA-3 (Touvron et al., 2023), Qwen (Bai et al., 2023), and ChatGLM3 (Du et al., 2022; Zeng et al., 2022). However, these LLMs remain too generalized to effectively adapt to specific QA tasks and may lack the capability to efficiently support continuous data and system updates in real-world applications. Existing pretrained models and fine-tuning methods have also shown inadequate results in our experiments. Additionally, developing a systematic approach to process enterprise-specific data remains an open challenge.

To address the aforementioned challenges, this paper proposes a comprehensive methodology tailored to the IT operations and maintenance domain for efficiently building enterprise-specific QA systems. We leverage LLMs and the concept of Retrieval Augmented Generation (RAG) framework, which enhances LLM performance (Gao et al., 2023). Following the standard RAG framework, our methodology consists of two components: (1) Offline Models Fine-tuning & Data Vectorization and (2) Online QA System Processing. This approach includes a generalized pipeline for structuring data from diverse, enterprise-specific sources, a training method for a domain-knowledge-augmented embedding model to capture unique semantics, and a supervised fine-tuning technique on an LLM to support varied QA tasks effectively.

More specifically, to build a QA system using our proposed methodology, the enterprise-specific corpora should first be collected and pre-processed. Through an automated pipeline for data cleaning, chunking, and distillation, we create a high-quality set of text chunks and

two annotated datasets for fine-tuning the embedding model and the LLM, respectively. To enhance task differentiation and model enterprise-specific semantics, we fine-tune the embedding model using contrastive learning (Gao et al., 2021) with Homogeneous In-Batch Negative Sampling (HIS) (Zhang et al., 2023) and Auxiliary Hard Negative Sampling (AHNS) strategies. The resulting text chunks are then embedded into vectors using the fine-tuned embedding model and stored in a vector database. For the LLM, we further fine-tune it with QA pairs using a Retrieval Augmented Fine-Tuning method. Details of this methodology can be found in the Data Processing Pipeline and Methodology sections.

In summary, the proposed methodology supports supervised fine-tuning on both enterprise-specific data and specialized QA tasks. Notably, due to the nature of the RAG mechanism, the vector database can be updated simply by adding new data, rather than requiring frequent re-fine-tuning of the LLM. The LLM can dynamically incorporate the top-k retrieved content from the database, consistently generating answers based on the latest and most relevant information. This approach enables continuous data and system updates at a low cost. We collected enterprise-specific corpora from the cloud computing domain, and experimental results demonstrate that our methodology outperforms comparable methods on QA tasks of varying complexity.

Key features of the proposed methodology include: (1) a general data processing pipeline for automatically handling multi-source, multi-format enterprise-specific corpora, (2) a fine-tuned embedding model for capturing enterprise-specific semantics, and (3) a fine-tuned generative LLM capable of supporting varied QA tasks. The methodology bridges the gap between abstract general theories and technologies like the RAG mechanism, pretrained models, and fine-tuning methods, and grounded customized solutions for specific enterprise applications. This work serves as a novel prototype between theory and grounded application.

The contributions of this paper include:

- To meet the growing demands on building enterprise-specific systems for complicated QA in IT operations and maintenance, we propose a comprehensive methodology based on the RAG mechanism.

This methodology is designed to support the modeling of private data and fine-tuning of LLMs.

- The methodology consists of two components: (1) Offline Models Fine-tuning & Data Vectorization, and (2) Online QA System Processing. We incorporate the latest techniques to fine-tune both the embedding model and the LLM, including contrastive learning with Homogeneous In-Batch Negative Sampling (HIN) and Auxiliary Hard Negative Sampling (AHNS) strategies, the use of instruction templates, and instruction tuning for LLMs-Continue Pre-Training (CPT) with a Retrieval Augmented Fine-Tuning (RAFT) approach.
- We collected real-world corpora related to IT operations and maintenance in cloud computing, and extensive experiments demonstrate that our methodology effectively improves performance across QA tasks of differing complexities. These results also highlight the potential of our approach for broader application in enterprise-level settings.

2. Related work

2.1. IT operations and maintenance

Traditionally, the quality of IT operations and maintenance varies greatly, as it largely depends on the personal experience of IT operators (Notaro et al., 2020). To support IT operators while managing the growing volume of IT-related information and knowledge, QA systems have become essential for improving efficiency across various application scenarios, leveraging advancements in NLP techniques (Elhoone et al., 2020; Huang et al., 2023a). These systems are designed to help IT operators quickly access relevant information and formulate troubleshooting plans (Rijal et al., 2022). However, in practice, IT operators may need to interact with QA systems multiple times to develop plans for complex tasks like troubleshooting, as current QA systems lack the intelligence to provide a comprehensive solution in a single interaction.

2.2. Data engineering and preprocessing

Effective data engineering and preprocessing are critical for training AI systems, including those in IT operations and maintenance. Many classic NLP techniques can be applied to structure data effectively. Recent advancements (Guo et al., 2024; Wu et al., 2024) have emphasized the importance of automated, context-aware frameworks for dataset construction. Recent findings also indicate that even common preprocessing steps (e.g., lowercasing, stopword removal) can cause substantial performance variation in both traditional and Transformer-based models, depending on dataset structure and task design (Anandarajan et al., 2019; Hickman et al., 2022; Siino et al., 2024). Additionally, recent studies (Hou et al., 2023; Long et al., 2024) highlight the need for advanced data engineering techniques in software engineering, particularly in synthetic data generation and prompt engineering, as essential for maintaining data quality and relevance. These approaches are crucial for optimizing LLM performance in specific contexts, ensuring models are well-suited to their intended tasks. However, general and systematic data processing methods for enterprise-specific tasks remain underexplored.

2.3. Large language models

Recent LLMs have shown substantial progress in open-domain QA tasks (Achiam et al., 2023; Brown et al., 2020). However, closed-source models like GPT-4, Claude, and Gemini cannot effectively address domain-specific or enterprise-exclusive questions, as they are not trained on private documents. Open-source models like LLaMA-3 (Touvron et al., 2023), Qwen (Bai et al., 2023), and ChatGLM3 (Du et al., 2022; Zeng et al., 2022) can be directly fine-tuned on specific corpora to provide QA services. Nonetheless, two major concerns remain. First, LLMs often produce hallucinated information (Guo et al., 2023), which

is unacceptable in industrial applications. Second, with the rapid growth of data, LLM-based QA systems require frequent re-fine-tuning, incurring significant costs. RAG frameworks offer a promising solution by addressing these issues and enhancing LLM-based QA systems. Recent efforts to develop domain-specific LLMs, such as OWL (Guo et al., 2024), show potential, yet they still struggle to adapt to grounded QA scenarios in real-world industrial IT operations.

2.4. Retrieval augmented generation

To address the limitations of LLMs regarding factual accuracy and domain-specific applications, the RAG framework has emerged as a promising approach (Gao et al., 2023). RAG techniques enhance LLM capabilities by incorporating relevant external information into input queries, thereby improving the accuracy and factual consistency of generated responses. In various domain-specific applications, RAG has proven highly effective for modeling domain-related semantics and enabling LLMs to produce factual and reliable answers (Gupta et al., 2024; Wang et al., 2024a; Zhang et al., 2024). Recent studies have further expanded RAG's potential by exploring fine-tuning methods for pre-trained LLMs specifically designed for RAG tasks (Lin et al., 2023; Wang et al., 2023a; Xu et al., 2023b; Zhang et al., 2024). This paper builds on the RAG concept, proposing a more comprehensive and practical RAG framework tailored to the IT operations and maintenance domain.

2.5. Prompt engineering

Prompt engineering has become a key technique for adapting LLMs to specific tasks without full-scale fine-tuning, focusing on crafting optimized input prompts to enhance model performance. Recent studies have proposed structured methods such as step-wise reasoning, iterative refinement, and role-based prompting to improve generation accuracy and robustness (Chen et al., 2025; Velásquez-Henao et al., 2023; Ye et al., 2024). In practical applications, prompt engineering has demonstrated effectiveness in reducing hallucinations, improving fairness, and enabling LLMs to address domain-specific tasks in fields like sexism detection, requirements engineering, clinical QA, and energy-aware scheduling (Arvidsson & Axell, 2023; Siino & Tinnirello, 2024a,b,c). Moreover, investigations into prompt consistency and reliability show that different prompt styles affect model outputs, especially in high-stakes domains like medicine (Marvin et al., 2024; Wang et al., 2024b). While prompt engineering has shown great benefits across domains, its integration into enterprise-specific QA workflows-particularly in complex IT operations scenarios-remains limited. Nevertheless, it presents a highly promising strategy for enabling cost-efficient, modular adaptation of LLMs to evolving enterprise needs. While still underexplored in enterprise-specific QA workflows, prompt engineering offers a cost-effective way to adapt LLMs. Our use of instruction templates reflects this potential and may support more flexible QA workflows in future work.

2.6. Parameter-efficient tuning techniques

While our framework primarily focuses on embedding optimization and RAG-based retrieval, recent developments in parameter-efficient fine-tuning (PEFT) techniques are also highly relevant to our goal of minimizing the cost of frequent model adaptation. Approaches such as prompt-tuning, prefix-tuning, adapters, and LoRA enable effective task alignment by tuning only a small number of parameters (Houlsby et al., 2019; Hu et al., 2022; Lester et al., 2021; Li & Liang, 2021). Building on these foundations, recent work proposes even more efficient strategies: LoRA aggregates ultra-low-rank updates for compact adaptation, SparseLoRA introduces contextual sparsity for faster training, and LoFT aligns low-rank updates with full fine-tuning behavior while preserving inference cost (Khaki et al., 2025; Quercia et al., 2025; Tastan et al.,

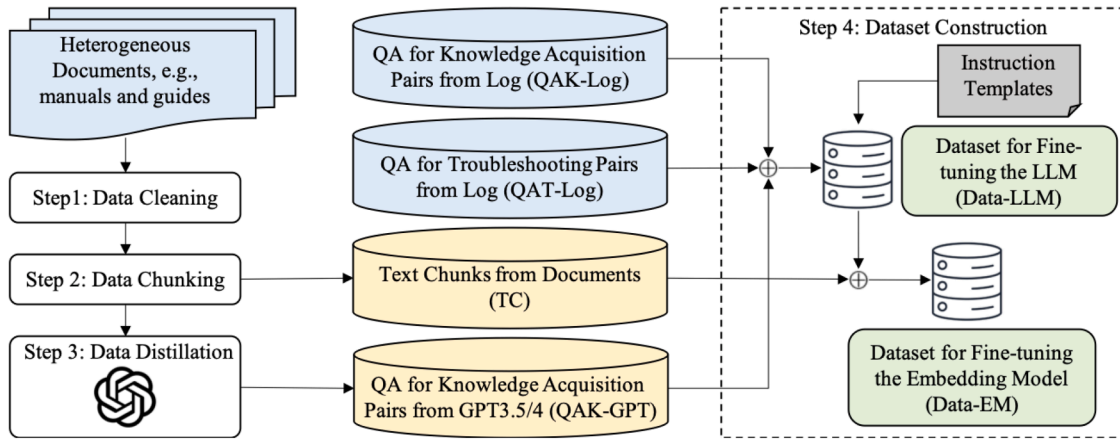


Fig. 2. Data processing pipeline: The datasets with a blue background represent raw enterprise-specific corpora collected from their original databases, those with a yellow background are preprocessed datasets, and those with a green background are constructed for the subsequent steps of model fine-tuning.

2025). These lightweight strategies are especially promising for enterprise use cases like IT operations, where LLMs must continuously adapt to evolving documentation without incurring full retraining overhead.

3. Data and processing pipeline

In this section, we present a general, systematic, and automated data processing pipeline for structuring enterprise-specific heterogeneous resources and materials. The pipeline consists of four main steps: data cleaning, data chunking, data distillation, and dataset construction. To our knowledge, this is the first comprehensive and automated solution designed to meet the broad requirements for building QA systems in the domain of IT operations and maintenance. Additionally, it incorporates an innovative data distillation step, leveraging the latest GPT-4 models.

3.1. Description of data in IT operations and maintenance

To build QA systems for IT operations and maintenance, enterprise-specific data must first be collected. Below, we outline the types of data typically required. Generally, this data can be obtained separately from IT operations and maintenance sources:

- **IT Operations Data:** This data is usually provided by the IT operations team within an enterprise and includes diverse documents (e.g., manuals and guides) and QA pairs. The documents are often in Word format and contain enterprise-specific knowledge, such as tool descriptions, operation examples, system configurations, and code scripts. They frequently include non-text elements like bullet points, numbered steps, and tables, which need to be removed.
- **Maintenance Data:** This data is typically provided by the maintenance team, comprising pairs of error logs and corresponding analyses. Error logs contain detailed descriptions of faults, functions, and platform-related information. The analyses, covering error scenarios, problem localization, and solutions, are either human-labeled or system-recorded. Problem localization generally includes function names, function descriptions, error reasons, priorities, and impacts.

3.2. Pipeline overview

As illustrated in Fig. 2, the raw enterprise-specific data are highlighted in blue and include diverse documents (e.g., manuals and guides), QA pairs for knowledge acquisition from logs (abbr. QAK-Log), and QA pairs for troubleshooting from logs (abbr. QAT-Log). It is worth noting that other types of QA data are also compatible with our pipeline.

The pipeline begins with *Step 1: Data Cleaning* and *Step 2: Data Chunking* applied to documents. Data cleaning includes a series of functions to

filter out unrelated content for tokenization, followed by data chunking, which divides longer text sequences into smaller segments, or “chunks.” This process produces a dataset of Text Chunks from Documents (abbr. TC). Next, *Step 3: Data Distillation* uses the GPT-3.5 or latest GPT-4 models to generate an additional dataset of QA pairs for knowledge acquisition (abbr. QAK-GPT), significantly expanding the QA dataset to enhance diversity, generalization, and contextual relevance. In *Step 4: Dataset Construction*, the QAK-Log, QAT-Log, and QAK-GPT datasets are integrated to construct a dataset (named with Data-LLM) for fine-tuning the generative LLM within the RAG framework. Additionally, by combining the Data-LLM and TC datasets, another dataset (named with Data-EM) is created for fine-tuning the embedding model in the RAG framework.

The proposed data processing pipeline is designed to fine-tune the embedding model and generative LLM within a RAG mechanism for building QA systems in IT operations and maintenance. The steps and methods used in the pipeline are applicable across various enterprises, as long as the objective is to develop a QA system using enterprise-specific data. The following subsections provide a detailed explanation of each step.

3.3. Step 1: Data cleaning

In the initial data cleaning step, all raw enterprise-specific data is converted into a clean text format. For example, tables from Word files are transformed into plain text following LaTeX standards, with each row separated by a line break and each column divided by a vertical bar. Additionally after tokenization, we standardize the text by removing non-text tokens and correcting invalid text tokens.

3.4. Step 2: Data chunking

Data chunking is necessary for the RAG mechanism, as the vector database has limitations on content length. Given the complexity of technical documents, this step is essential for creating coherent segments. Our chunking approach prioritizes preserving the logical flow within each sentence. Instead of setting a fixed chunk length, we aim to retain the complete meaning and structure of the content as much as possible, particularly in tables.

When extracting information from Word documents, each line is categorized by type, including content, style, and font. The style indicates whether the text is a title or body content and specifies the title level. Since each title represents a distinct section, we segment the content according to title hierarchy. An algorithm is used to extract chunks by progressively breaking down from higher-level titles to lower levels.

Table 1

Instruction template designed for data distillation in the processing pipeline.

Assume you are the IT operation team member and you have some questions to inquire.
 Assume the following document can answer your question. What questions and corresponding answers can you post?
 Please post as many knowledge based questions as possible.
 Do not post the question without an answer.
 Answer should be complete and must be got from the document.
 Question with very long answer is allowed.
 If you cannot find any question or cannot provide answer, please respond <unk>.
 Use <sep> to connect each QA. Content: <content>

3.5. Step 3: Data distillation

To enhance the QA dataset's diversity, generalization, and contextual relevance for subsequent model fine-tuning, we include a data distillation step using the latest GPT-3.5/4 models.¹

For each chunk extracted from the documents, a QA pair is generated via GPT-3.5/4 APIs, guided by an instruction template as shown in Table 1. To manage costs, GPT-3.5 is used by default to generate multiple QAs per chunk, with the number of generations $i = \text{round}(2 + \frac{n-1000}{500})$, where n is the character count of the chunk. This adaptive generation captures more details in longer, complex text. If GPT-3.5 does not produce a response in the required format, GPT-4 is used to ensure accuracy. This distillation process results in the QAK-GPT dataset, containing approximately 1.6k QA pairs. All the generated data are adopted without any operation for filtering out bad cases, because we expect to lower manual cost for data reviewing and see how this easy strategy can affect on the final performance. Even so, we evaluate the generated data by randomly selecting samples and find that more than 90% of them are valid.

This approach enables the construction of a dataset with varied questions corresponding to identical answers, allowing the model to generalize more effectively to semantically similar questions during training.

3.6. Step 4: Dataset construction

Using the processed TC and QAK-GPT datasets, along with the raw enterprise-exclusive log data (QAK-Log and QAT-Log), we construct two fine-tuning datasets—Data-LLM for the generative LLM and Data-EM for the embedding model within the RAG framework, as illustrated in the right section of Fig. 2. Given their shared format, the original datasets can be seamlessly combined.

4. Methodology

To streamline the data modeling and model fine-tuning processes for QA systems in IT operations and maintenance, we propose a novel framework detailed in this section. The framework comprises two main components: offline model fine-tuning and data vectorization, and the online QA system process based on the RAG mechanism.

4.1. Offline component: Models fine-tuning & data vectorization

Using the preprocessed text chunks and datasets, the embedding model and generative LLM within the RAG framework are fine-tuned to better accommodate enterprise-specific semantics and QA tasks. As illustrated in Figs. 3 and 4, the Data-EM dataset is utilized for fine-tuning the embedding model, while the Data-LLM dataset is used for fine-tuning the LLM. Once fine-tuned, the embedding model generates vectorized representations of all text data, which are stored in a vector database for the retrieval-augmented generation tasks in the online component.

4.1.1. Fine-tuning the embedding model

In fine-tuning the embedding model, we use the Dense Passage Retrieval (DPR) framework (Karpukhin et al., 2020) as our core retrieval approach. DPR leverages embedding models to create dense vector representations of queries and passages, supporting efficient and precise retrieval. Starting with the pretrained embedding model BGE-M3 (Xiao et al., 2023), valued for its compact structure and strong performance on the MTEB benchmark (Muennighoff et al., 2022), we further optimized retrieval performance through contrastive learning, utilizing two negative sampling strategies to sharpen the model's ability to discern domain-relevant from non-relevant passages.

Homogeneous in-batch sampling (HIS). To strengthen the discriminative power of embeddings, a large set of negative samples is essential (Qu et al., 2020; Wang et al., 2022). Although in-batch negative sampling is commonly used to supply numerous negative samples, it has limitations in our context: Negative samples across diverse tasks may not accurately capture semantic relationships within a specific task context. To address this, we organize each mini-batch to include training data from only the same task, ensuring homogeneity among in-batch negatives and enhancing their effectiveness in refining embedding discrimination. Our approach integrates both in-batch and hard negatives and employs cross-device sharing (Xiao et al., 2021) to increase the pool of accessible negative samples.

Auxiliary hard negative sampling (AHNS). Given an IT operations and maintenance dataset $\mathcal{X}$, our goal is to define an encoding function $f : \mathcal{X} \rightarrow \mathbb{R}^d$ that maps each document chunk or question $x_i \in \mathcal{X}$ to a position within a d -dimensional embedding space. The aim is for embeddings of related chunks and questions (x_i, x'_i) to be close, while embeddings of unrelated ones remain distant. For a random subset (or batch) of N positive pairs $\mathcal{X}_N = \{(\tilde{x}_i, \tilde{x}'_i)\}_{i=1}^N$, where $\tilde{x}_i$ and $\tilde{x}'_i$ denote a document chunk and its corresponding question, we define the contrastive loss function for the encoder f as follows:

$$\mathcal{L}_{x_i} = -\log \frac{\exp(s(\tilde{x}_i, \tilde{x}'_i)/\tau)}{\exp(s(\tilde{x}_i, \tilde{x}_i)/\tau) + \sum_{\tilde{x}_j \in \mathcal{X}_N} \exp(s(\tilde{x}_i, \tilde{x}_j)/\tau)}, \quad (1)$$

where $s(x_i, x_j) = \frac{f(x_i)^\top f(x_j)}{\|f(x_i)\| \|f(x_j)\|}$ denotes the inner product of the normalized latent representations of x_i and x_j , and τ is a temperature scaling hyperparameter. $\tilde{x}_i$ is the positive sample associated with $\tilde{x}'_i$, while all other instances $\tilde{x}_j \neq \tilde{x}_i \in \mathcal{X}_N$ are treated as negative samples. Our goal is to select high-quality, informative hard negative examples from this set. Typically, negative examples are chosen through random sampling (Chen et al., 2020a,b). In our approach, we use the DPR framework with an initial embedding model to retrieve the top-k relevant chunks for each positive sample. We then designate all remaining chunks, excluding the actual document chunks, as hard negative samples.

4.1.2. Fine-tuning the generative LLM

For fine-tuning the generative LLM, we use the state-of-the-art LLM Qwen-14b-Base (Bai et al., 2023) as the backbone. To train the generative LLM for various QA tasks, we design instruction templates that format each example in Data-LLM. The details of these templates for each QA task are provided in Table 2. Additionally, we employ two training methods to further enhance the LLM's performance.

Continue pre-training. With the preprocessed domain-specific datasets, we aim to imbue the Qwen-14b-base model with specialized knowledge in IT operations and maintenance, thereby enhancing its ability to understand and generate relevant content within this domain. This method follows the standard approach outlined in Gururangan et al. (2020).

Retrieval augmented fine-tuning method. To further improve the LLM's ability to leverage retrieved information for IT operations and maintenance tasks, we implement a retrieval-augmented fine-tuning approach.

¹ <https://chatgpt.com/>

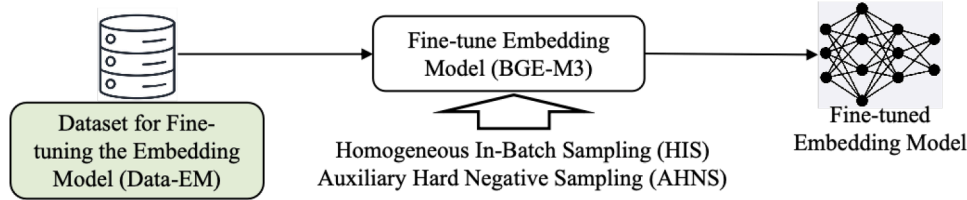


Fig. 3. Process of fine-tuning the embedding model with HIS and AHNS methods.

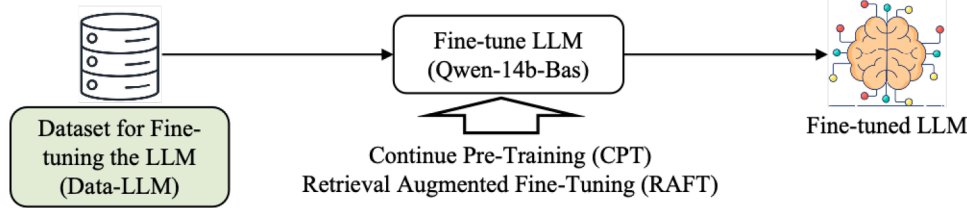


Fig. 4. Process of fine-tuning the LLM with CPT and RAFT methods.

Table 2

Instruction templates designed for fine-tuning the generative LLM for different QA tasks.

The instruction template for LLM fine-tuning on knowledge acquisition task:
Assume you are a customer service representative, and you have received a question from a user or the operations team:
Content: <content>
Please answer the user's question concisely and professionally based on the following known information:
Content: <content>
The instruction template for LLM fine-tuning on troubleshooting task:
Please conduct a root cause analysis of the sudden AIOPS event based on the error log below. The analysis should include: 1. Scenario, 2. Problem localization (including service, method name, function, keywords, event type, event level, impact scope), 3. Solution (including personnel involved and resolution plan). Content: <content>
Below is a historical case:
Content: <content>

Based on the Data-LLM dataset, we construct an extended training set $D = \{(x^{(i)} \circ I^{(i)}, y^{(i)})\}_{i=1}^M$, where $x^{(i)} \circ I^{(i)}$ represents an input query $x^{(i)}$ combined with retrieved chunks $I^{(i)}$, and $y^{(i)}$ denotes the corresponding output answer. For each example $(x^{(i)}, y^{(i)}) \in D$, we retrieve the top-k relevant text chunks $I^{(i)} \subset C$ based on $x^{(i)}$. The fine-tuning instances are then created by pairing each retrieved chunk with the question, using an instruction template detailed in Table 2. The objective function for this supervised instruction tuning is defined as follows:

$$\mathcal{L}_m = -\frac{1}{N} \sum_{i=1}^N \mathbb{E}_{x,y,I \in D_i} \log P(y^{(i)} | I^{(i)} \circ x^{(i)}), \quad (2)$$

where $P(y^{(i)} | I^{(i)} \circ x^{(i)})$ represents the probability of generating the correct output $y^{(i)}$ given the input $x^{(i)}$ augmented with the retrieved chunks $I^{(i)}$. This approach offers two key benefits: it enables the LLM to leverage relevant and up-to-date background knowledge, and it improves the LLM's ability to generate factual answers.

Our QA generator is fine-tuned via supervised instruction tuning using LoRA, which offers a tradeoff between parameter efficiency and adaptability. Alternative low-rank or prompt-based tuning strategies (e.g., prompt-tuning, prefix-tuning) are not explored in this work but remain promising directions for future optimization.

4.1.3. Data vectorization

Once the fine-tuned embedding model is obtained, a vector database can be constructed. All enterprise-specific data is embedded into low-dimensional vectors via the embedding model, and these vectors are

stored in the database. Fig. 5 illustrates the data vectorization process. Notably, the database can be easily updated by inserting or modifying data, without the need for frequent re-fine-tuning of the embedding model. With the embedding model, generative LLM, and vector database prepared, they can be assembled to create the RAG-based QA system.

4.2. Online component: Runtime QA system process

In the online stage of building a runtime QA system, as shown in Fig. 6, IT operators can use the system and pose a question. The fine-tuned embedding model then converts the question into an embedding, which is used to retrieve relevant content from the vector database. We use FAISS (Johnson et al., 2019), a library for efficient similarity search, to identify the most relevant document chunks. The retrieved information, along with the question, is formatted using the instruction template to construct the input prompt for the LLM. Finally, the LLM generates an answer by referencing all the content in the input prompt. This entire process follows the standard RAG mechanism, ensuring efficient response times.

5. Experiment

5.1. Dataset

The raw enterprise-exclusive data and the datasets processed through our data pipeline are summarized in Table 3. Using these datasets, the embedding model and LLM can be fine-tuned, enabling the development of a RAG-based QA system. To evaluate our proposed method, we also collected a dataset called Data-Eval. This dataset consists of 319 questions created by domain experts, including 236 questions for the knowledge acquisition task and 83 for the troubleshooting task. Each question is paired with relevant chunks from the enterprise-exclusive corpora, and all questions are accompanied by labeled answers.

5.2. Baselines

We consider the following popular text embedding models as the baselines for our embedding model evaluation: GTE-large-zh, BGE-M3, Text2Vec-base, M3E-base, jina-embeddings-v2-base-zh, and BGE-large-zh-v1.5. To evaluate the generative LLM, our method are compared with several state-of-the-art language models: Chatglm3-6b, Qwen-7b-Chat, Llama3-8B-Instruct, and Qwen-14b-Chat.

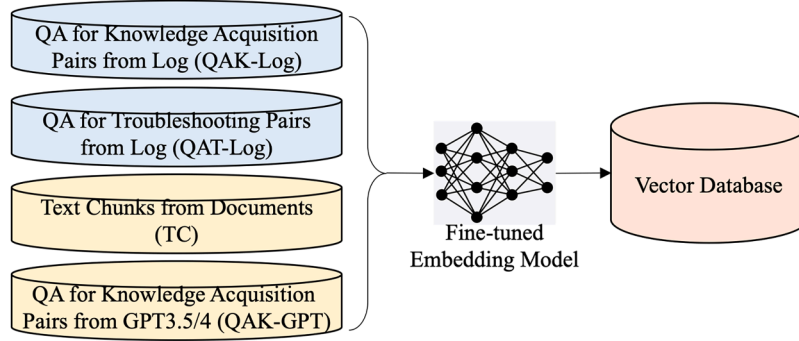


Fig. 5. Process of data vectorization.

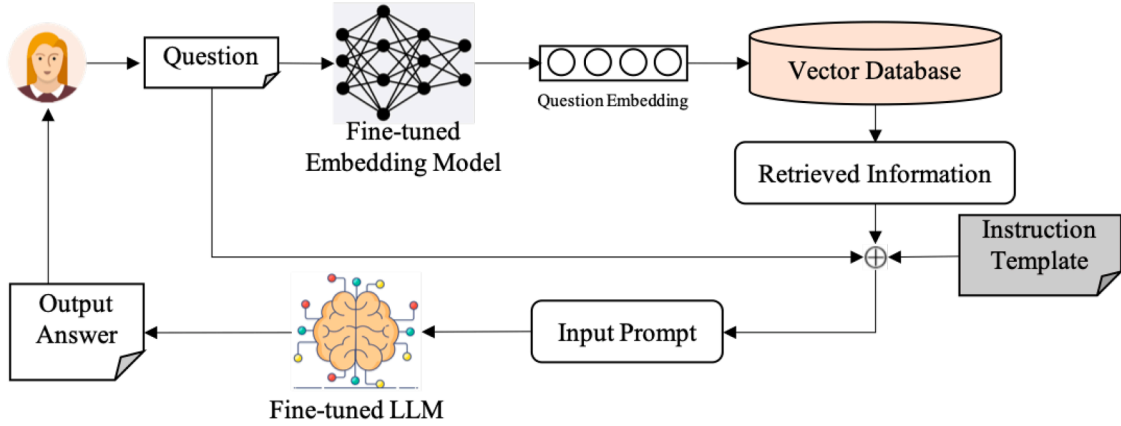


Fig. 6. Process of runtime QA system based on standard RAG mechanism. The details of prompt and template are present in the section of Appendix.

5.3. Metrics

To evaluate the effectiveness of the embedding model in the knowledge acquisition and troubleshooting tasks, we assess performance using top-k retrieval accuracy (Acc@K). The formal definition of Acc@K is as follows: $R(q, C) \rightarrow \hat{C}$ takes a question q and a set of chunks C , returning a smaller set $\hat{C}$, where $\hat{C} \subseteq C$ and $|\hat{C}| = k \ll |C|$. Top-k retrieval accuracy measures the fraction of questions for which $\hat{C}$ contains a span that can answer the question. In our experiments, we present separate results for top-k retrieval with k set to 1, 5, or 20.

To evaluate the performance of the generative LLM, we use two evaluation methods: single-score mode and pairwise-score mode (Guo et al., 2024; Huang et al., 2024; Xu et al., 2023a; Zheng et al., 2024). In single-score mode, we select the model to be tested and generate answers based on given questions and fixed reference chunks, using the BGE-M3 embedding model as the default. We employ both GPT-4 (Achiam et al., 2023) and GPT-5 as scoring models to evaluate the generated responses. Each response is rated on a scale from 1 to 10, with higher scores indicating better quality and we set 6 as the pass score. To ensure reliability, we run the scoring model three times for each case and report the average score. In pairwise-score mode, both models generate answers to identical questions using the same reference chunks. A scoring model then assesses which model's response is superior, assigning a win to the better performer and a loss to the other. If the performances are comparable, both models receive a tie. Detailed prompts and procedures for both evaluation modes are provided in Tables B.1 and B.2 of Appendix section.

5.4. Experimental settings

When fine-tuning the embedding model, we set the learning rate to 10^{-6} and a batch size of 1024. The Adam optimizer is used with $\beta_1 = 0.9$

and $\beta_2 = 0.99$. We also implemented a homogeneous in-batch sampling strategy, where all samples within the same batch come from the same task, and employed cross-device negatives to increase the diversity of negative samples. The model was trained for 1 epoch using 8 * A100 80G GPUs.

For continuing pre-training the LLM, the learning rate is set to 2×10^{-5} with a weight decay of 0.1 and a global batch size of 128. The sequence length is set to 2048. We again use the Adam optimizer with $\beta_1 = 0.9$ and $\beta_2 = 0.99$. The training lasts for 3 epochs.

For retrieval-augmented fine-tuning, the learning rate is increased to 5×10^{-5} , maintaining the same weight decay of 0.1. The global batch size is increased to 512, and the sequence length remains at 2048. Adam is used once more as the optimization algorithm, with the same β values. This phase is trained for 1 epoch. We perform full parameter training for continued pre-training, and LoRA (Hu et al., 2021) fine-tuning for retrieval-augmented fine-tuning, consistently using 8 * A100 80G GPUs.

5.5. Evaluation results for embedding model

In Table 4, our domain-knowledge-augmented embedding model outperforms the baseline models across both tasks. Specifically, for the QA for Knowledge Acquisition task (QA for KA), our full model with Homogeneous In-Batch Sampling (HIS) and Auxiliary Hard Negative Sampling (AHNS) achieves the highest Acc@5 and Acc@20 scores of 0.919 and 0.979, respectively, outperforming the BGE-M3 baseline by 4.3% and 2.2% on these metrics. In the QA for Troubleshooting task (QA for TS), our full model also delivers the best performance, achieving the highest scores across all metrics: Acc@1 of 0.759, Acc@5 of 0.795, and Acc@20 of 0.795. These results represent improvements of 16.6%, 4.7%, and 1.5%, respectively, over the BGE-M3 baseline.

Additionally, we evaluated the inference time of our embedding model on an A100 80G GPU, as shown in Table 5, with the results

Table 3
Statistics of the datasets used in our framework.

Dataset Name	Dataset Description	Indicator	Number
TC	Text Chunks	Chunks #	3824
		Avg. token #	529
QAK-Log	QA pairs for knowledge acquisition from log	Sample #	1468
		Avg. question token #	16
		Avg. answer token #	56
QAK-GPT	QA pairs for knowledge acquisition	Sample #	16,973
		Avg. question token #	15
		Avg. answer token #	66
QAT-Log	QA pairs for troubleshooting from log	Sample #	47,471
		Avg. question token #	235
		Avg. answer token #	370
Data-EM	Dataset for fine-tuning the embedding model	Sample #	65,912
Data-LLM	Dataset for fine-tuning the LLM	Sample #	65,912
		Avg. question token #	1233
		Avg. answer token #	186
Data-Eval	Dataset for evaluation	Sample #	319
		Avg. question token #	53
		Avg. answer token #	133

Table 4
Comparison of the fine-tuned embedding model with baselines. Acc@K represents top-K retrieval accuracy.

Method	Supported Max Length	QA for Knowledge Acquisition			QA for Troubleshooting		
		Acc@1	Acc@5	Acc@20	Acc@1	Acc@5	Acc@20
Text2Vec-base (Xu, 2023)	512	0.314	0.496	0.606	0.735	0.771	0.771
M3E-base (Wang et al., 2023b)	512	0.305	0.572	0.758	0.639	0.735	0.771
GTE-large-zh (Li et al., 2023)	512	0.487	0.708	0.822	0.554	0.687	0.747
BGE-large-zh-v1.5 (Xiao et al., 2023)	512	0.525	0.767	0.902	0.602	0.723	0.735
jina-embeddings-v2-base-zh (Mohr et al., 2024)	8192	0.369	0.674	0.847	0.566	0.747	0.783
BGE-M3 (Chen et al., 2024)	8192	0.610	0.881	0.958	0.651	0.759	0.783
Ours	8192	0.661	0.919	0.979	0.759	0.795	0.795

Table 5
Response time (ms) for once retrieval by using our fine-tuned embedding model.

QA for KA			QA for TS		
Chunks@1	Chunks@5	Chunks@20	Chunks@1	Chunks@5	Chunks@20
15.4	30.9	46.7	27.7	55.6	84.2

demonstrating the efficiency of our method. For real-world applications, we focus on $k = 5$, which balances latency and accuracy. Table 5 also shows that higher k-values (e.g., $k = 20$) increase response time, making them impractical for deployment. These results of $k = 20$ are provided for reference.

5.6. Evaluation results for LLM

In single-score mode, we compared our proposed model to several baseline models, including Chatglm3-6b, Qwen-7b-Chat, Llama3-8B-Instruct, and Qwen-14b-Chat. As shown in Table 6, our model with Continue Pre-Training (CPT) and Retrieval-Augmented Fine-Tuning (RAFT) achieves the highest mean scores of 6.68 and 6.88 in both the QA for Knowledge Acquisition and QA for Troubleshooting tasks. These scores represent improvements of 0.63 and 0.29 points, respectively, over the Qwen-14b-Chat baseline. Results by using GPT-5 are reported in Table 7,

and the consistent performance improvements across both evaluators (i.e., GPT-4 and GPT-5) further confirm the robustness of our method.

In pairwise scoring (see Fig. 7), our model also outperforms all baselines in both tasks.

5.7. Evaluation results for end-to-end system

To address concerns about the system's practical performance, we conducted an experiment to assess its end-to-end performance. The results are listed in Table 8. The end-to-end and individual component evaluation results are strongly correlated, meaning that improvements to either the embedding model or the LLM lead to enhanced overall performance.

5.8. Ablation study

The ablation study experiment is two-fold. For the embedding model, we evaluated the impact of HIS and AHNS. The results in Table 9 show that both techniques contribute to performance improvements, with their combination yielding the best results across all metrics in both tasks.

For the LLM, we conducted an ablation study to examine the importance of CPT and RAFT. In Table 10, supervised fine-tuning without chunks (i.e., w/o CPT, w/o RAFT) showed improvements over its original performance without fine-tuning, as shown in Table 6. CPT along

Table 6

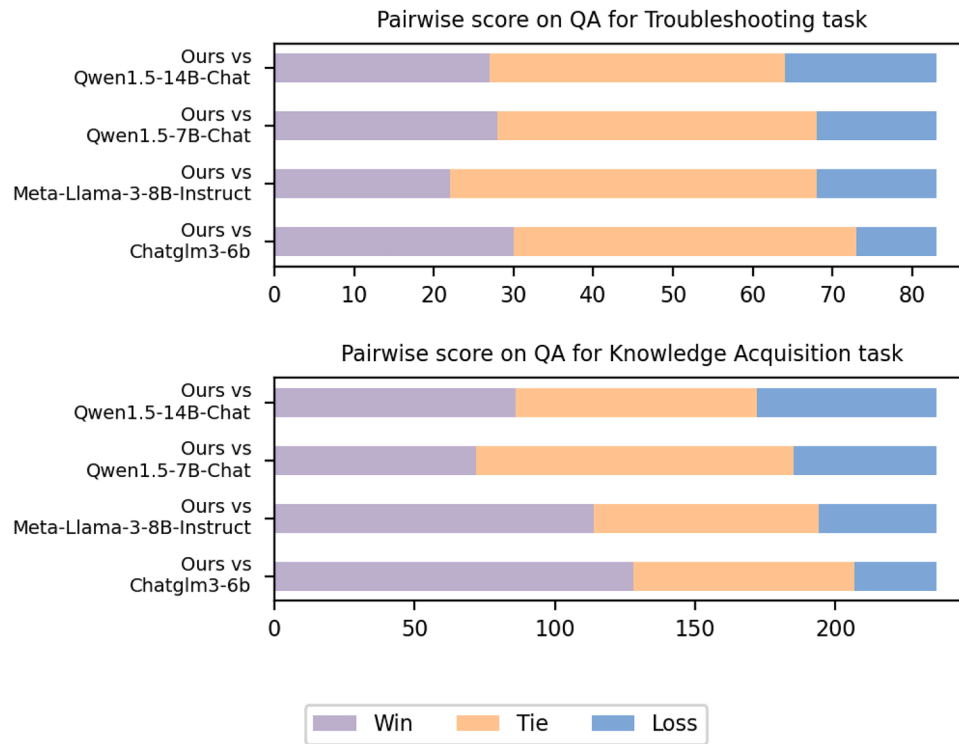
Results of single-score mode evaluation on the fine-tuned LLM. Score1-3 mean that the GPT-4 are called for three times to evaluate each case.

Method	QA for Knowledge Acquisition				QA for Troubleshooting			
	Score1	Score2	Score3	Mean	Score1	Score2	Score3	Mean
Chatglm3-6b (Du et al., 2022)	5.19	5.28	5.20	5.22	4.01	4.06	4.26	4.11
Qwen-7b-Chat (Bai et al., 2023)	5.89	5.84	5.80	5.84	5.21	5.37	5.22	5.27
Llama3-8B-Instruct (Touvron et al., 2023)	5.32	5.23	5.40	5.32	5.61	5.68	5.62	5.64
Qwen-14b-Chat (Bai et al., 2023)	6.57	6.58	6.63	6.59	5.99	6.07	6.10	6.05
Ours	6.92	7.01	6.70	6.88	6.72	6.65	6.68	6.68

Table 7

Results of single-score mode evaluation on the fine-tuned LLM using GPT-5 as the scoring model. Each case was evaluated three times, and we report Score1-3 along with their mean values.

Method	QA for Knowledge Acquisition				QA for Troubleshooting			
	Score1	Score2	Score3	Mean	Score1	Score2	Score3	Mean
Chatglm3-6b (Du et al., 2022)	4.51	4.46	4.49	4.49	3.15	3.11	3.11	3.12
Qwen-7b-Chat (Bai et al., 2023)	5.32	5.22	4.92	5.15	4.65	4.50	4.76	4.64
Llama3-8B-Instruct (Touvron et al., 2023)	4.36	4.41	4.53	4.43	4.95	4.75	4.87	4.86
Qwen-14b-Chat (Bai et al., 2023)	5.91	5.87	5.82	5.87	4.52	4.43	4.42	4.46
Ours	6.09	6.28	6.19	6.19	5.74	5.76	5.71	5.74

**Fig. 7.** Pairwise comparison of our LLM against baselines in two tasks, evaluated by GPT-4.**Table 8**

Results of end-to-end evaluation on both the fine-tuned embedding model and LLM. The metrics are consistent with Table 6.

Embedding Model	LLM	QA for Knowledge Acquisition				QA for Troubleshooting			
		Score1	Score2	Score3	Mean	Score1	Score2	Score3	Mean
BGE-M3	Qwen-14b-Chat	6.57	6.58	6.63	6.59	5.99	6.07	6.10	6.05
BGE-M3	Ours	6.92	7.01	6.70	6.88	6.72	6.65	6.68	6.68
Ours	Qwen-14b-Chat	6.73	6.68	6.69	6.70	6.13	6.17	6.15	6.15
Ours	Ours	7.03	7.10	7.05	7.06	6.77	6.75	6.70	6.74

Table 9

Ablation study results for the fine-tuned embedding model.

HIS	AHNS	QA for KA			QA for TS		
		Acc@1	Acc@5	Acc@20	Acc@1	Acc@5	Acc@20
$\times$	$\times$	0.661	0.895	0.970	0.711	0.771	0.790
$\checkmark$	$\times$	0.650	0.903	0.974	0.735	0.783	0.795
$\times$	$\checkmark$	0.665	0.915	0.970	0.721	0.783	0.795
$\checkmark$	$\checkmark$	0.661	0.919	0.979	0.759	0.795	0.795

Table 10

Ablation study results for the fine-tuned LLM.

CPT	RAFT	QA for KA				QA for TS			
		Score1	Score2	Score3	Mean	Score1	Score2	Score3	Mean
$\times$	$\times$	6.57	6.65	6.61	6.61	6.15	6.10	6.08	6.11
$\checkmark$	$\times$	6.62	6.61	6.68	6.64	6.14	6.15	6.10	6.13
$\times$	$\checkmark$	6.66	6.63	6.72	6.67	6.58	6.75	6.62	6.65
$\checkmark$	$\checkmark$	6.92	7.01	6.70	6.88	6.72	6.65	6.68	6.68

((w CPT, w/o RAFT)) improved the mean scores from 6.61 to 6.64 for KA QA and from 6.11 to 6.13 for TS QA. RAFT alone (w/o CPT, w/ RAFT) further improved the mean scores to 6.67 and 6.65, outperforming standard supervised fine-tuning and demonstrating its effectiveness in enhancing model performance, which means only adding RAFT could be more effective than only adding CPT. In a summary, our full model, incorporating both CPT and RAFT, achieved the highest mean scores of 6.88 for Knowledge Acquisition and 6.68 for Troubleshooting. This represents significant improvements of 0.27 points (4.1%) for Knowledge Acquisition and 0.57 points (9.3%) for Troubleshooting compared to the model (w/o CPT, w/o RAFT), highlighting the complementary benefits of our proposed techniques. Admittedly, adding a component will not always improve the performance, e.g., the Score3 on the KA task and Score2 on the TS task. We guess this is perhaps due to the limited training data volume and the model training process could be unstable. We would leave this investigation as a future work.

5.9. Discussion

By summary, existing RAG systems for domain-specific applications always differ along three axes: (1) Retriever capacity-from generic encoders to domain-adapted embedding learners and two-stage retrieve-then-rerank pipelines; (2) Generator coupling-from vanilla in-context RAG to retrieval-augmented fine-tuning (RAFT) and continued pre-training on domain corpora; and (3) Operational constraints-context length, end-to-end latency, and robustness to noisy enterprise data (e.g., logs and terminologies). Our work positions itself at the intersection of these axes: We (1) emphasize domain-specific embedding learning via HIS + AHNS, (2) pair CPT + RAFT with task-specific instruction templates for knowledge acquisition and troubleshooting, and (3) explicitly design for production latency (favoring small-k retrieval) with general and grounded data construction pipeline method before model training. This combination targets enterprise-specific IT operations and maintenance constraints that are under-addressed by prior general-purpose RAG variants.

6. Conclusion and future work

In this paper, we present a comprehensive methodology for building tailored QA systems in the domain of IT operations and maintenance, based on the standard RAG mechanism. First, we developed a general data processing pipeline that includes data cleaning, chunking, distilla-

tion, and dataset construction using raw enterprise-specific data. With the processed datasets, an embedding model is fine-tuned using Homogeneous In-Batch Negative Sampling and Auxiliary Hard Negative Sampling strategies, improving the model's retrieval performance. Additionally, we fine-tuned a generative LLM with well-designed instruction templates to enhance its ability to handle domain-specific QA tasks through Continue Pre-Training and Retrieval Augmented Fine-Tuning. We evaluated our methodology through a series of experiments, assessing its performance on distinct QA tasks with varying levels of difficulty. The results demonstrate the effectiveness of the proposed approach in IT operations and maintenance.

In the future, some promising directions are valuable to be further investigated. (1) The applicability and adaptation of our method to other domains can be explored; (2) Besides, parameter-efficient tuning methods such as prefix-tuning or adapters to further reduce deployment costs without compromising domain adaptability can be comprehensively investigated; (3) Other advanced RAG frameworks can be experimented, comparing their performance with the proposed method based on standard RAG mechanism.

CRedit authorship contribution statement

Zhuoxuan Jiang: Conceptualization, Formal analysis, Funding acquisition, Methodology, Project administration, Supervision, Validation, Visualization, Writing—original draft; **Tianyang Zhang:** Conceptualization, Data Curation, Formal analysis, Methodology, Investigation, Resources, Software, Validation, Writing—review & editing; **Shengguang Bai:** Data Curation, Methodology, Investigation, Validation, Writing—review & editing; **Lin Lin:** Writing—review & editing; **Haotian Zhang:** Writing—review & editing; **Yinong Xun:** Writing—review & editing; **Jiawei Ren:** Project administration, Writing—review & editing; **Wen Si:** Project administration, Writing—review & editing; **Shaohua Zhang:** Project administration, Writing—review & editing.

Data availability

The authors do not have permission to share data.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is supported by the [Shanghai Science International Cooperation Project](#), China (Grant No. 24170790602), the Program of Shanghai Committee of Science and Technology, China (Grant No. 24511103204), the Natural Science Foundation of Shanghai (Grant No. 25ZR1401273), and the 2024 special program of Artificial Intelligence Promotion on the Reform of Scientific Research Paradigms of Shanghai Committee of Education, China.

Appendix A. Case study

To provide insight into real-world applications of our framework, we present two representative cases and the answers are generated by our method. [Table A.1](#) illustrates a QA scenario for knowledge acquisition, and [Table A.2](#) demonstrates another QA scenario for troubleshooting in IT operations and maintenance.

Table A.1

Example of an answer generated by our method for knowledge acquisition task in IT operations and maintenance.

User questions:

In the fault handling process for localized service anomalies, what are the specific steps for collecting exception information?

Core model input:

Assume you are a customer service representative, and you have received a question from a user or the operations team:

In the fault handling process for localized service anomalies, what are the specific steps for collecting exception information?

Please answer the user's question concisely and professionally based on the following known information:

Segment 0: Internal customer document Segment 1

Segment 1: Internal customer document Segment 2

Segment 2: Internal customer document Segment 3

Segment 3: Internal customer document Segment 4

Segment 4: Internal customer document Segment 5

System response:

Specific steps for collecting exception information include:

Organize effective information based on customer complaints/internal alerts: The dimensions are not limited to the following information; the more comprehensive the information, the better.

Attempt to analyze/reproduce the problem: Based on the information provided by the customer, manually designate nodes for testing. Simultaneously, use the internal platform Grafana to investigate quality data across multiple dimensions such as bandwidth, status codes, and response times by combining domain name, node, and time information.

Table A.2

Example of an answer generated by our method for troubleshooting task in IT operations and maintenance.

User questions:

Please help analyze and resolve the following error: GetCustomBiz query for custom topology failed. The error is: Querying business: *'s custom topology node id: [] failed, context canceled, project_space = * namespace.name = *.

Core model input:

Please conduct a root cause analysis of the sudden AIOps event based on the error log below. The analysis should include: 1. Scenario, 2. Problem localization (including service, method name, function, keywords, event type, event level, impact scope), 3. Solution (including personnel involved and resolution plan).

GetCustomBiz query for custom topology failed. The error is: Querying business: *'s custom topology node id: [] failed, context canceled, project_space = * namespace.name = *.

Below is a historical case:

Segment 0: Error log case and solution 1

Segment 1: Error log case and solution 2

Segment 2: Error log case and solution 3

Segment 3: Error log case and solution 4

Segment 4: Error log case and solution 5

System response:

Scenario: * log error

Project: *

Service: *

Method name: *

Function description: Set custom business NodeName for clusters and modules

Key information: GetCustomBiz query for custom topology failed. The error is querying custom topology node failed, topology node id: []

Event type: Query * timeout caused query failure

Event level: *

Impact scope: Unable to normally obtain custom business NodeName, which may affect cluster and other interface queries

Solution:

Based on the error information, the query for custom topology node failed because the topology node id is empty. Possible solutions are as follows:

1. Check if the topology node id parameter is correctly passed. Ensure that the passed topology node id is not empty and is in the expected format.
2. Check if the topology node data exists. Confirm that the data corresponding to the topology node id exists in the system and has not been deleted or modified.
3. Check the logic of querying the custom topology. Ensure that the code logic for querying the custom topology is correct and there are no missed or erroneous handling of the topology node id.
4. Check the status of * services. If querying the custom topology requires other services, such as *, ensure these services are running normally and can be accessed properly.
5. Check logs and error information. View more detailed error logs and error information to further locate the problem. It may be necessary to increase log recording or adjust the log level to obtain more detailed information.

Appendix B. Instruction templates for evaluation

The detailed instruction templates used by LLMs in evaluation are shown in [Table B.1](#) for knowledge acquisition and [Table B.2](#) for troubleshooting.

Table B.1

Prompts used for pairwise-score and single-score modes evaluation on knowledge acquisition task.

Pairwise-Score Mode Prompt:
Please act as an impartial evaluator and assess the quality of answers provided by two AI assistants to a user's question. Your evaluation should consider the correctness and helpfulness of the answers. You will be given a reference answer, Assistant A's answer, and Assistant B's answer. Your task is to determine which assistant's answer is better. Evaluation steps: 1. Compare both assistants' answers to the reference answer. 2. Identify and correct any errors in the assistants' answers. 3. Avoid any positional bias, ensuring that the order of the answers does not influence your decision. 4. Do not let the length of the answers affect your assessment. 5. Please answer based on facts, expressing the required information for the question. 6. Do not favor certain assistant names. Be as objective as possible. After providing your explanation, please output your final verdict in the following JSON format: <pre> “json { "verdict": "Can only be A or B or Tie", "explanation": "Your explanation" } ” </pre>
Single-score Mode Prompt:
Please act as an impartial evaluator and assess the quality of an answer provided by an AI assistant to a user's question. We will provide a question, a corresponding reference answer, and the assistant's answer. Your evaluation should consider the correctness of the answer. Evaluation steps: 1. Please compare the assistant's answer to the reference answer. 2. Identify and correct any errors. 3. Evaluate as objectively as possible, paying attention to factual errors in the assistant's answer that are not present in the reference answer. 4. If the assistant does not address the content of the reference answer, it will be scored as 0. After providing your explanation, you must rate the answer on a scale of 1 to 10 using the following JSON format: <pre> “json { "rating": "1 to10//, "explanation": "Your explanation" } ” </pre>

Table B.2

Prompts for pairwise-score and single-score modes evaluation on troubleshooting task.

Pairwise-Score Mode Prompt:
Please act as an impartial evaluator and assess the quality of answers provided by two AI assistants to a user's question. Your evaluation should consider the correctness and helpfulness of the answers. You will be given a reference answer, Assistant A's answer, and Assistant B's answer. Your task is to determine which assistant's answer is better. Evaluation steps: 1. Compare both assistants' answers to the reference answer. 2. Identify and correct any errors in the assistants' answers. 3. Avoid any positional bias, ensuring that the order of the answers does not influence your decision. 4. Do not let the length of the answers affect your assessment. 5. Please answer based on facts, expressing the required information for the question. 6. Do not favor certain assistant names. Be as objective as possible. 7. The reference answer includes 7 fields, each field is worth 1 point, with the solution field worth 4 points. Please strictly compare the answers of both assistants for each field and analyze them. Based on the field scores, determine which assistant's answer is better, or if it's a tie After providing your explanation, please output your final verdict in the following JSON format: <pre> “json { "verdict": "Can only be A or B or Tie", "explanation": "Your explanation" } ” </pre>
Single-score Mode Prompt:
Please act as an impartial evaluator and assess the quality of an answer provided by an AI assistant to a user's question. We will provide a question, a corresponding reference answer, and the assistant's answer. Your evaluation should consider the correctness of the answer. Evaluation steps: 1. Please compare the assistant's answer to the reference answer. 2. Identify and correct any errors. 3. Evaluate as objectively as possible, paying attention to factual errors in the assistant's answer that are not present in the reference answer. 4. If the assistant does not address the content of the reference answer, it will be scored as 0. 5. The reference answer includes 7 fields, each field is worth 1 point, with the solution field worth 4 points. For each field in the assistant's answer, please strictly score according to the field score. If the answer is inaccurate or incorrect for a field, no points should be awarded for that field. After providing your explanation, you must rate the answer on a scale of 1 to 10 using the following JSON format: <pre> “json { "rating": "1 to10//, "explanation": "Your explanation" } ” </pre>

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altschmidt, J., Altman, S., Anadkat, S. et al. (2023). GPT-4 technical report. *arXiv: 2303.08774*.
- Anandarajan, M., Hill, C., & Nolan, T. (2019). *Practical text analytics*. Springer Cham.
- Arvidsson, S., & Axell, J. (2023). Prompt engineering guidelines for LLMs in requirements engineering. Bachelor thesis University of Gothenburg and Chalmers University of Technology, Department of Computer Science and Engineering Gothenburg, Sweden.
- Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., Fan, Y., Ge, W., Han, Y., Huang, F. et al. (2023). Qwen technical report. *arXiv: 2309.16609*.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A. et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Chen, B., Zhang, Z., Langrené, N., & Zhu, S. (2025). Unleashing the potential of prompt engineering for large language models. *Patterns*, 6(6), 101260.
- Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., & Liu, Z. (2024). BGE M3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv: 2004.10964*.
- Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020a). A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th international conference on machine learning* (pp. 1597–1607).
- Chen, X., Fan, H., Girshick, R., & He, K. (2020b). Improved baselines with momentum contrastive learning. *arXiv: 2003.04297*.
- Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S. et al. (2023). PaLM: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240), 1–113.
- Du, M., Li, F., Zheng, G., & Srikanth, V. (2017). DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security* (pp. 1285–1298).
- Du, Z., Qian, Y., Liu, X., Ding, M., Qiu, J., Yang, Z., & Tang, J. (2022). GLM: General language model pretraining with autoregressive blank infilling. In *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: Long papers)* (pp. 320–335).
- Elhoone, H., Zhang, T., Anwar, M., & Desai, S. (2020). Cyber-based design for additive manufacturing using artificial neural networks for industry 4.0. *International Journal of Production Research*, 58(9), 2841–2861.
- Galup, S. D., Dattero, R., Quan, J. J., & Conger, S. (2009). An overview of IT service management. *Communications of the ACM*, 52(5), 124–127.
- Gao, T., Yao, X., & Chen, D. (2021). SimCSE: Simple contrastive learning of sentence embeddings. *arXiv: 2104.08821*.
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv:2312.10997*.
- Guo, H., Guo, Y., Yang, J., Liu, J., Li, Z., Zheng, T., Zheng, L., Hou, W., & Zhang, B. (2023). LogLG: Weakly supervised log anomaly detection via log-event graph construction. In *Proceedings of international conference on database systems for advanced applications* (pp. 490–501).
- Guo, H., Yang, J., Liu, J., Yang, L., Chai, L., Bai, J., Peng, J., Hu, X., Chen, C., Zhang, D., Shi, X., Zheng, T., Zheng, L., Zhang, B., Xu, K., & Li, Z. (2024). OWL: A large language model for IT operations. In *Proceedings of the 12th international conference on learning representations* (pp. 1–24).
- Gupta, A., Shrigaonkar, A., de Luis, B. A., Silva, B., Holstein, D., Li, D., Marsman, J., Nunes, L. O., Rouzbahman, M., Sharp, M. et al. (2024). Rag vs fine-tuning: Pipelines, tradeoffs, and a case study on agriculture. *arXiv: 2401.08406*.
- Gururangan, S., Marasović, A., Swayamdipta, S., Lo, K., Beltagy, I., Downey, D., & Smith, N. A. (2020). Don't stop pretraining: Adapt language models to domains and tasks. *arXiv: 2004.10964*.
- Hickman, L., Thapa, S., Tay, L., Cao, M., & Srinivasan, P. (2022). Text preprocessing for text mining in organizational research: Review and recommendations. *Organizational Research Methods*, 25(1), 114–146.
- Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., & Wang, H. (2023). Large language models for software engineering: A systematic literature review. *arXiv: 2308.10620*.
- Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., De Laroussilhe, Q., Gesmundo, A., Attariyan, M., & Gelly, S. (2019). Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th international conference on machine learning* (pp. 2790–2799).
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). Lora: Low-rank adaptation of large language models. *arXiv: 2106.09685*.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *Proceedings of the 10th international conference on learning representations* (pp. 1–13).
- Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B. et al. (2023a). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv: 2311.05232*.
- Huang, S., Liu, Y., Fung, C., Qi, J., Yang, H., & Luan, Z. (2023b). LogQA: Question answering in unstructured logs. *arXiv: 2303.11715*.
- Huang, Y., Bai, Y., Zhu, Z., Zhang, J., Zhang, J., Su, T., Liu, J., Lv, C., Zhang, Y., Fu, Y. et al. (2024). C-Eval: A multi-level multi-discipline chinese evaluation suite for foundation models. *Advances in Neural Information Processing Systems*, 36, 1–20.
- Jäntti, M., & Cater-Steel, A. (2017). Proactive management of IT operations to improve IT services. *Journal of Information Systems and Technology Management*, 14(2), 191–218.
- Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547.
- Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 conference on empirical methods in natural language processing* (pp. 6769–6781).
- Khaki, S., Li, X., Guo, J., Zhu, L., Plataniotis, K. N., Yazdanbakhsh, A., Keutzer, K., Han, S., & Liu, Z. (2025). SparseloRA: Accelerating LLM fine-tuning with contextual sparsity. In *Proceedings of the 42nd international conference on machine learning* (pp. 29768–29783).
- Lester, B., Al-Rfou, R., & Constant, N. (2021). The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 conference on empirical methods in natural language processing* (pp. 3045–3059).
- Li, X. L., & Liang, P. (2021). Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing (volume 1: Long papers)* (pp. 4582–4597).
- Li, Z., Zhang, X., Zhang, Y., Long, D., Xie, P., & Zhang, M. (2023). Towards general text embeddings with multi-stage contrastive learning. *arXiv: 2308.03281*.
- Lin, X. V., Chen, X., Chen, M., Shi, W., Lomeli, M., James, R., Rodriguez, P., Kahn, J., Szilvasy, G., Lewis, M. et al. (2023). RA-DIT: Retrieval-augmented dual instruction tuning. *arXiv: 2310.01352*.
- Liu, Y., Tao, S., Meng, W., Yao, F., Zhao, X., & Yang, H. (2024). Logprompt: Prompt engineering towards zero-shot and interpretable log analysis. In *Proceedings of the 2024 IEEE/ACM 46th international conference on software engineering: Companion proceedings* (pp. 364–365).
- Long, L., Wang, R., Xiao, R., Zhao, J., Ding, X., Chen, G., & Wang, H. (2024). On LLMs-driven synthetic data generation, curation, and evaluation: A survey. *arXiv: 2406.15126*.
- Marvin, G., Hellen Raudha, N., Jjingo, D., & Nakatumba-Nabende, J. (2024). Prompt engineering in large language models. In *Data intelligence and cognitive informatics* (pp. 387–402).
- Mohr, I., Krimmel, M., Sturua, S., Akram, M. K., Koukounas, A., Günther, M., Mastrapas, G., Ravishankar, V., Martínez, J. F., Wang, F. et al. (2024). Multi-task contrastive learning for 8192-token bilingual text embeddings. *arXiv:2402.17016*.
- Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2022). MTEB: Massive text embedding benchmark. *arXiv: 2210.07316*.
- Notaro, P., Cardoso, J., & Gerndt, M. (2020). A systematic mapping study in AIOps. In *Proceedings of international conference on service-oriented computing* (pp. 110–123).
- Qu, Y., Ding, Y., Liu, J., Liu, K., Ren, R., Zhao, W. X., Dong, D., Wu, H., & Wang, H. (2020). RocketQA: An optimized training approach to dense passage retrieval for open-domain question answering. *arXiv: 2010.08191*.
- Quercia, A., Cao, Z., Bangun, A., Paul, R. D., Morrison, A., Assent, I., & Scharr, H. (2025). 1LoRA: Summation compression for very low-rank adaptation. *arXiv:2503.08333*.
- Rijal, L., Colomo-Palacios, R., & Sánchez-Gordón, M. (2022). AIOps: A multivocal literature review. *Artificial Intelligence for Cloud and Edge Computing*, Springer International Publishing, (pp. 31–50).
- Siino, M., & Tinnirello, I. (2024a). GPT Hallucination detection through prompt engineering. In *Conference and labs of the evaluation forum* (pp. 712–721).
- Siino, M., & Tinnirello, I. (2024b). GPT Prompt engineering for scheduling appliances usage for energy cost optimization. In *2024 IEEE International symposium on measurements & networking (m&n)* (pp. 1–6).
- Siino, M., & Tinnirello, I. (2024c). Prompt engineering for identifying sexism using GPT mistral 7B. In *Conference and labs of the evaluation forum* (pp. 1228–1236).
- Siino, M., Tinnirello, I., & La Cascia, M. (2024). Is text preprocessing still worth the time? A comparative survey on the influence of popular preprocessing methods on transformers and traditional classifiers. *Information Systems*, 121, 102342.
- Tastan, N., Laskaridis, S., Takac, M., Nandakumar, K., & Horvath, S. (2025). LoFT: Low-rank adaptation that behaves like full fine-tuning. *arXiv: 2505.21289*.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S. et al. (2023). Llama 2: Open foundation and fine-tuned chat models. *arXiv: 2307.09288*.
- Velásquez-Henao, J., Franco, C., & Cadavid, L. (2023). Prompt engineering: A methodology for optimizing interactions with AI-language models in the field of engineering. *DYNA*, 90(230), 9–17.
- Wang, B., Ping, W., McAfee, L., Xu, P., Li, B., Shoyebi, M., & Catanzaro, B. (2023a). Instructretro: Instruction tuning post retrieval-augmented pretraining. *arXiv: 2310.07713*.
- Wang, H., Huang, W., Deng, Y., Wang, R., Wang, Z., Wang, Y., Mi, F., Pan, J. Z., & Wong, K.-F. (2024a). UniMS-RAG: A unified multi-source retrieval-augmented generation for personalized dialogue systems. *arXiv: 2401.13256*.
- Wang, L., Chen, X., Deng, X., Wen, H., You, M., Liu, W., Li, Q., & Li, J. (2024b). Prompt engineering in consistency and reliability with the evidence-based guideline for LLMs. *NPJ Digital Medicine*, 7(1), 41.
- Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, D., Majumder, R., & Wei, F. (2022). Text embeddings by weakly-supervised contrastive pre-training. *arXiv: 2212.03533*.
- Wang, Y., Sun, Q., & He, S. (2023b). M3E: Moka massive mixed embedding model. <https://github.com/wangyuxinwhy/uniem>.
- Wu, S., Huang, Y., Gao, C., Chen, D., Zhang, Q., Wan, Y., Zhou, T., Zhang, X., Gao, J., Xiao, C. et al. (2024). UniGen: A unified framework for textual dataset generation using large language models. *arXiv: 2406.18966*.
- Xiao, S., Liu, Z., Shao, Y., Lian, D., & Xie, X. (2021). Matching-oriented product quantization for Ad-hoc retrieval. *arXiv: 2104.07858*.
- Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2023). C-Pack: Packaged resources to advance general Chinese embedding. *arXiv: 2309.07597*.
- Xu, C., Guo, D., Duan, N., & McAuley, J. (2023a). Baize: An open-source chat model with parameter-efficient tuning on self-chat data. *arXiv: 2304.01196*.
- Xu, M. (2023). Text2vec: Text to vector toolkit. <https://github.com/shibing624/text2vec>.

- Xu, P., Ping, W., Wu, X., McAfee, L., Zhu, C., Liu, Z., Subramanian, S., Bakhturina, E., Shoyebi, M., & Catanzaro, B. (2023b). Retrieval meets long context large language models. [arXiv: 2310.03025](#).
- Ye, Q., Axmed, M., Pryzant, R., & Khani, F. (2024). Prompt engineering a prompt engineer. [arXiv: 2311.05661](#).
- Zeng, A., Liu, X., Du, Z., Wang, Z., Lai, H., Ding, M., Yang, Z., Xu, Y., Zheng, W., Xia, X. et al. (2022). GLM-130B: An open bilingual pre-trained model. [arXiv: 2210.02414](#).
- Zhang, P., Xiao, S., Liu, Z., Dou, Z., & Nie, J.-Y. (2023). Retrieve anything to augment large language models. [arXiv: 2310.07554](#).
- Zhang, T., Patil, S. G., Jain, N., Shen, S., Zaharia, M., Stoica, I., & Gonzalez, J. E. (2024). RAFT: Adapting language model to domain specific rag. [arXiv: 2403.10131](#).
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. et al. (2024). Judging LLM-as-a-judge with MT-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36, 46595–46623.